Université de Lorraine - Master 2 Sciences des Données (M2SDD)

Projet commun : Algorithmique · Programmation · Mathématiques pour l'informatique

Année universitaire 2025-2026

Projet Réussite

Dossier d'Analyse

Prédiction de la réussite d'un apprenant
à partir de ses traces numériques sur la plateforme ARCHE

Auteur : Matthieu PELINGRE
Email : matthieu.pelingre1@etu.univ-lorraine.fr
Dépôt : github.com/M2SDD/projet-reussite
Date : Avril 2026

Données : exports anonymisés de la plateforme ARCHE
Bibliothèques : Python 3 · pandas · scikit-learn · scipy · matplotlib

Table des matières

1 Problématique et contexte **3**
1.1 La plateforme ARCHE . . . 3
1.2 Problématique . . . 3
1.3 Approche retenue . . . 3

2 Données – ETL (Extraction, Transformation, Chargement) **4**
2.1 Sources de données . . . 4
2.1.1 Fichier de logs . . . 4
2.1.2 Fichier de notes . . . 4
2.2 Statistiques descriptives des données brutes . . . 5
2.3 Chargement et validation (DataLoader) . . . 5
2.4 Nettoyage des données (DataCleaner) . . . 5
2.4.1 Suppression des doublons . . . 5
2.4.2 Jointure et gestion des valeurs manquantes . . . 5

3 Feature Engineering **6**
3.1 Principe . . . 6
3.2 Familles de variables . . . 6
3.2.1 Métriques d'activité globale . . . 6
3.2.2 Patterns temporels . . . 6
3.2.3 Variables par composant ARCHE (`comp_*`) . . . 6
3.2.4 Variables par type d'événement (`evt_*`) . . . 7
3.2.5 Régularité de connexion . . . 7
3.2.6 Profondeur d'interaction . . . 7
3.3 Bilan . . . 7

4 Sélection de variables **8**
4.1 Pré-filtrage . . . 8
4.1.1 Filtre de variance quasi-nulle . . . 8
4.1.2 Filtre de colinéarité . . . 8
4.2 Méthodes de sélection combinées . . . 8

5 Modélisation **10**
5.1 Standardisation des données . . . 10
5.2 Modèle 1 : Régression Linéaire Multiple (RLM) . . . 10
5.3 Modèle 2 : Random Forest (Forêt Aléatoire) . . . 10
5.4 Modèle 3 : Gradient Boosting . . . 11

6 Évaluation et comparaison des modèles **12**
6.1 Métriques de performance . . . 12
6.2 Résultats obtenus . . . 12
6.3 Visualisations . . . 12

6.4 Analyse des résultats 13

7 Discussion, limites et perspectives **14**
7.1 Pourquoi ces résultats sont-ils décevants ? 14
7.1.1 Taille insuffisante du jeu de données 14
7.1.2 Questionnement sur la pertinence des variables 14
7.1.3 Absence d'optimisation des hyperparamètres 14
7.1.4 Contraintes techniques 14
7.2 Une classification aurait peut-être été plus adaptée 15
7.3 Pistes d'amélioration 15

Conclusion **16**

Chapitre 1

Problématique et contexte

1.1 La plateforme ARCHE

ARCHE est la plateforme numérique de l'Université de Lorraine, basée sur le logiciel libre **Moodle**. Elle centralise les ressources pédagogiques et enregistre automatiquement les interactions des apprenants sous forme de *logs* : chaque clic, consultation de document, soumission de devoir ou participation au forum génère une entrée horodatée.

1.2 Problématique

Est-il possible de prédire la réussite d'un apprenant en analysant ses traces numériques au sein de la plateforme ARCHE ?

Plus précisément, il s'agit de comparer la **régression linéaire multiple** avec au moins une autre technique d'apprentissage automatique, et de proposer une solution motivée par les résultats d'évaluation.

1.3 Approche retenue

Le pipeline d'analyse suit les quatre étapes du projet :

1. **ETL :** chargement et nettoyage des deux fichiers CSV sources ;
2. **Feature Engineering :** calcul d'indicateurs comportementaux à partir des logs bruts, puis sélection des variables les plus pertinentes ;
3. **Modélisation :** entraînement de trois modèles de régression (régression linéaire multiple, Random Forest, Gradient Boosting) ;
4. **Évaluation :** comparaison des modèles sur les données de test.

Remarque

La démarche CRISP-DM (*Cross Industry Standard Process for Data Mining*) aurait idéalement guidé ce projet. Cependant, l'interdiction des notebooks Jupyter a rendu difficile la phase d'analyse exploratoire des données interactive, généralement réalisée dans ce format. Ce dossier tente de restituer la démarche analytique suivie malgré cette contrainte.

Chapitre 2

Données – ETL (Extraction, Transformation, Chargement)

2.1 Sources de données

Deux fichiers CSV ont été fournis, exportés de façon anonymisée :

Fichier	Rôle
`logs_info_25_pseudo.csv`	Traces d'activité
`notes_info_25_pseudo.csv`	Variable cible

2.1.1 Fichier de logs

Chaque ligne représente **un événement** généré par un utilisateur de la plateforme. Le fichier contient les colonnes suivantes :

Colonne	Type	Description
`heure`	datetime	Horodatage de l'événement (`YYYY-MM-DD HH:MM:SS`)
`pseudo`	int	Identifiant anonyme de l'utilisateur
`contexte`	str	Ressource concernée par l'activité
`composant`	str	Type de composant ARCHE (Fichier, Forum, Quiz...)
`evenement`	str	Détail de l'action (cours consulté, module soumis...)

2.1.2 Fichier de notes

Chaque ligne correspond à **un étudiant évalué** :

Colonne	Type	Description
`pseudo`	int	Identifiant anonyme (clé de jointure)
`note`	float	Note obtenue (sur 20)

2.2 Statistiques descriptives des données brutes

Caractéristiques du jeu de données

- **Logs :** 16 227 événements au total
- **Notes :** 99 étudiants évalués
- **Moyenne des notes :** $\approx$ 10,00 / 20
- **Médiane des notes :** $\approx$ 9,86 / 20
- **Seuil de risque défini :** $<$ 10 pts

Hétérogénéité des populations

Le fichier de logs ne contient pas uniquement des étudiants : des enseignants, administrateurs et autres acteurs de la plateforme y figurent également (non présents dans les notes). De plus, certains étudiants notés n'apparaissent pas dans les logs (absence totale).
La stratégie adoptée (jointure `inner`) supprime ces observations non appariées pour éviter d'introduire du bruit dans les données d'apprentissage.

2.3 Chargement et validation (DataLoader)

La classe `DataLoader` assure :

1. le chargement en UTF-8 (nécessaire pour les accents des exports ARCHE) ;
2. la validation de la présence de toutes les colonnes attendues ;
3. la conversion stricte de la colonne `heure` en type `datetime64[ns]` avec détection des valeurs invalides (converties en `NaT`) ;
4. un avertissement non bloquant si des colonnes supplémentaires sont détectées.

2.4 Nettoyage des données (DataCleaner)

2.4.1 Suppression des doublons

Des lignes strictement identiques peuvent apparaître dans les exports ARCHE (double-clics, re-soumissions). La méthode `remove_duplicates()` utilise `DataFrame.drop_duplicates` sur l'ensemble des colonnes.

2.4.2 Jointure et gestion des valeurs manquantes

La jointure entre le tableau de features (indexé sur `pseudo`) et le tableau de notes s'effectue avec un `inner join` par défaut (option activable dans l'interface). Les rares valeurs manquantes résiduelles sont remplacées par zéro.

Chapitre 3

Feature Engineering

3.1 Principe

Le feature engineering est l'étape centrale du projet : il s'agit de **transformer les logs bruts** (une ligne par événement) en un **tableau récapitulatif par étudiant** (une ligne par étudiant, avec plusieurs dizaines de variables calculées).

Ces variables cherchent à capturer différentes dimensions du comportement d'apprentissage en ligne : volume d'activité, régularité, type d'interactions, répartition horaire, etc.

3.2 Familles de variables

3.2.1 Métriques d'activité globale

Variable	Description
`actions_totales`	Nombre total d'événements générés
`jours_actifs`	Nombre de jours calendaires distincts avec au moins un événement
`duree_engagement_jours`	Jours entre la première et la dernière activité
`nombre_sessions`	Sessions de travail (seuil inter-session : 30 minutes)

3.2.2 Patterns temporels

Variable	Description
`heure_pointe`	Heure de la journée la plus fréquente (mode)
`ratio_activite_weekend`	Fraction des événements survenant en week-end
`actions_matin`	Événements entre 6h et 12h
`actions_aprem`	Événements entre 12h et 18h
`actions_soir`	Événements entre 18h et 24h
`actions_nuit`	Événements entre 0h et 6h

3.2.3 Variables par composant ARCHE (`comp_*`)

Un **tableau croisé** (*crosstab*) est construit entre les étudiants et les composants ARCHE utilisés (Fichier, Forum, Quiz, Devoir, etc.). Chaque colonne `comp_<composant>` contient le nombre d'événements de ce type pour l'étudiant considéré.

3.2.4 Variables par type d'événement (`evt_*`)

Même approche avec la colonne `evenement` : chaque type d'action (cours consulté, module soumis, commentaire créé, etc.) devient une variable numérique.

3.2.5 Régularité de connexion

Variable	Description
`jours_consecutifs_max`	Longueur maximale d'une série de jours actifs consécutifs
`ecart_moyen_jours`	Écart moyen (en jours) entre deux jours actifs successifs
`ecart_type_jours`	Variabilité de cet écart (régularité du rythme)
`jours_actifs_hebdos`	Nombre moyen de jours actifs par semaine

3.2.6 Profondeur d'interaction

Variable	Description
`actions_par_jour`	Ratio actions / jours actifs
`diversite_composants`	Nombre de composants distincts utilisés
`diversite_contextes`	Nombre de ressources distinctes consultées
`interactions_moy_par_composant`	Moyenne des actions par composant
`taux_changement_composant`	Fréquence de changement de composant entre deux actions consécutives

3.3 Bilan

L'extraction produit plusieurs dizaines de variables par étudiant. Le nombre exact dépend des composants et événements présents dans les données, les colonnes `comp_*` et `evt_*` étant dynamiques.

Chapitre 4

Sélection de variables

4.1 Pré-filtrage

Deux filtres sont appliqués avant toute sélection avancée.

4.1.1 Filtre de variance quasi-nulle

Utilise `VarianceThreshold(threshold=0.01)` de scikit-learn. Supprime les variables dont la quasi-totalité des valeurs sont identiques, typiquement des composants ARCHE que personne ou presque n'utilise.

4.1.2 Filtre de colinéarité

Calcule la matrice de corrélation de Pearson (valeur absolue). Pour chaque paire de variables dont $|r| > 0{,}85$, la variable à la variance la plus faible est supprimée.

Ce filtre est crucial car le feature engineering génère naturellement des variables fortement corrélées entre elles. Par exemple, `actions_totales` et `actions_par_jour` sont très liées pour les étudiants ayant des durées d'engagement similaires.

4.2 Méthodes de sélection combinées

Trois méthodes ont été combinées pour la sélection finale lors du test du logiciel :

Méthode	Justification
`linear` (Test F)	Mesure la corrélation linéaire avec la note. Adapté à la régression linéaire.
`mutual_info`	Capture les relations non linéaires entre variables et cible. Complémentaire du Test F.
`rfe` (RFE)	Élimination récursive avec Random Forest (lent). Tient compte des interactions entre variables.

L'union des $k = 15$ variables sélectionnées par chaque méthode est retenue, garantissant que ni le modèle linéaire ni les modèles d'ensemble ne sont désavantagés par la sélection.

Le dataset résultant comporte **95 échantillons** et **22 variables**, divisés en 76 observations d'entraînement et 19 de test (ratio 80 % / 20 %).

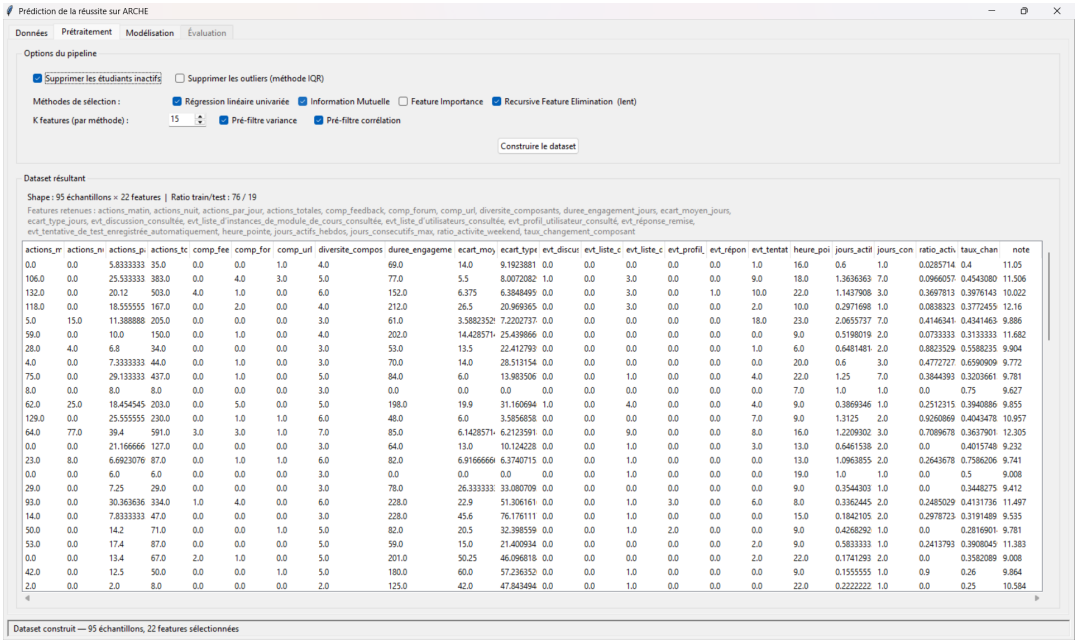

Figure 4.1 : Onglet *Prétraitement* : Configuration du pipeline et dataset résultant

Variables sélectionnées

D'après la session illustrée dans l'onglet Prétraitement, les 22 variables retenues incluent notamment : `actions_matin`, `actions_nuit`, `actions_totales`, plusieurs colonnes `comp_*`, `diversite_composants`, `diversite_contextes`, `duree_engagement_jours`, `jours_actifs`, `jours_consecutifs_max`, `nombre_sessions`, `ratio_activite_weekend`, etc.

Chapitre 5

Modélisation

5.1 Standardisation des données

Avant l'entraînement de tout modèle, les variables sont standardisées à l'aide d'un `RobustScaler` :

$$X_{\text{scaled}} = \frac{X - \text{médiane}(X)}{\text{IQR}(X)} \tag{5.1}$$

Le `RobustScaler` a été préféré au `StandardScaler` car les données de logs présentent de fortes disparités. La médiane et l'écart interquartile (IQR) sont robustes aux valeurs extrêmes, contrairement à la moyenne et à l'écart-type.

Le scaler est entraîné uniquement sur les données d'entraînement puis appliqué aux données de test, pour éviter toute fuite d'information (*data leakage*).

5.2 Modèle 1 : Régression Linéaire Multiple (RLM)

La régression linéaire multiple modélise la note y comme une combinaison linéaire des variables sélectionnées $x_1, \ldots, x_p$:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p \tag{5.2}$$

Les coefficients β_i sont estimés par la méthode des **moindres carrés ordinaires** (MCO) :

$$\hat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \tag{5.3}$$

Ce modèle suppose notamment la linéarité de la relation, la normalité des résidus (vérifiable via le test de Shapiro-Wilk) et l'absence de multicolinéarité, partiellement traitée par le pré-filtrage en amont.

5.3 Modèle 2 : Random Forest (Forêt Aléatoire)

La forêt aléatoire agrège les prédictions de B arbres de décision :

$$\hat{y}_{\text{RF}} = \frac{1}{B} \sum_{b=1}^{B} T_b(\mathbf{x}) \tag{5.4}$$

Paramètre	Valeur	Rôle
`n_estimators`	200	Nombre d'arbres
`max_depth`	10	Profondeur maximale
`max_features`	`sqrt`	Variables candidates par noeud
`min_samples_split`	5	Min. d'échantillons pour diviser
`bootstrap`	False	Sans remise (petit jeu de données)
`criterion`	`friedman_mse`	Critère de division

5.4 Modèle 3 : Gradient Boosting

Le Gradient Boosting construit les arbres séquentiellement, chacun corrigeant les erreurs du précédent :

$$\hat{y}^{(m)} = \hat{y}^{(m-1)} + \eta \cdot T_m(\mathbf{x}) \tag{5.5}$$

où η est le taux d'apprentissage et T_m le m-ième arbre résiduel.

Paramètre	Valeur	Rôle
`n_estimators`	100	Nombre d'itérations (arbres)
`learning_rate`	0,1	Taux d'apprentissage
`max_depth`	3	Arbres peu profonds
`subsample`	0,8	Sous-échantillonnage stochastique

Chapitre 6

Évaluation et comparaison des modèles

6.1 Métriques de performance

Trois métriques principales ont été calculées sur les données de test (19 observations) :

R^2 (coefficient de détermination) Proportion de la variance de la note expliquée par le modèle. Un $R^2 = 1$ correspond à une prédiction parfaite ; un $R^2 = 0$ revient à prédire la moyenne ; un $R^2 < 0$ signifie que le modèle est moins bon que la prédiction naïve.

RMSE (*Root Mean Squared Error*) Erreur quadratique moyenne, exprimée en points de note. Pénalise davantage les erreurs importantes.

MAE (*Mean Absolute Error*) Erreur absolue moyenne, exprimée en points de note. Plus robuste aux erreurs extrêmes.

6.2 Résultats obtenus

Table 6.1 : Comparaison des performances sur les données de test (19 observations)

Modèle	R^2	RMSE	MAE
Régression Linéaire	−0,4260	1,2073	0,9431
Random Forest	+0,1262	0,9451	0,7828
Gradient Boosting	−0,2601	1,1349	0,8710

6.3 Visualisations

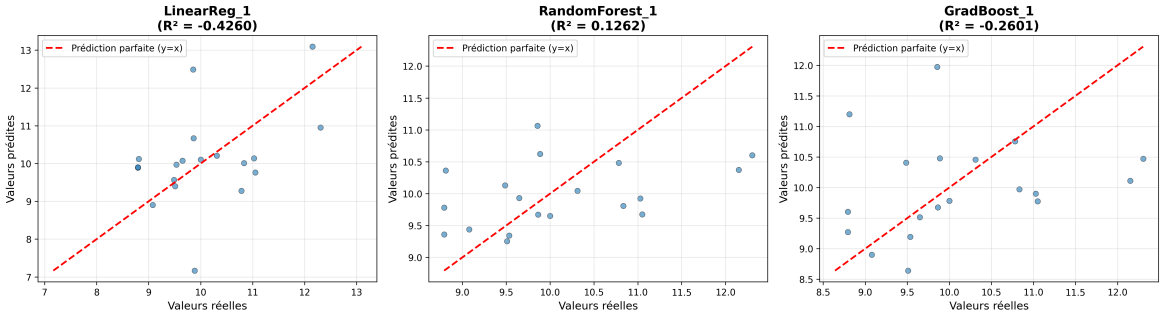

Figure 6.1 : Prédictions vs valeurs réelles pour les trois modèles

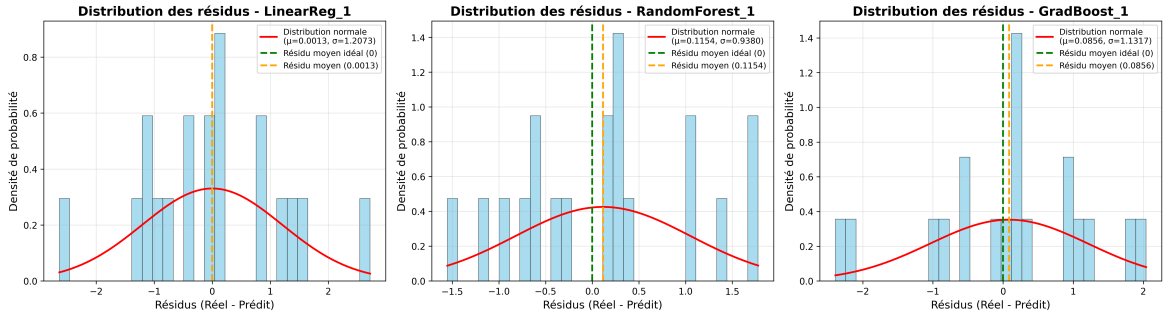

Figure 6.2 : Distribution des résidus pour les trois modèles

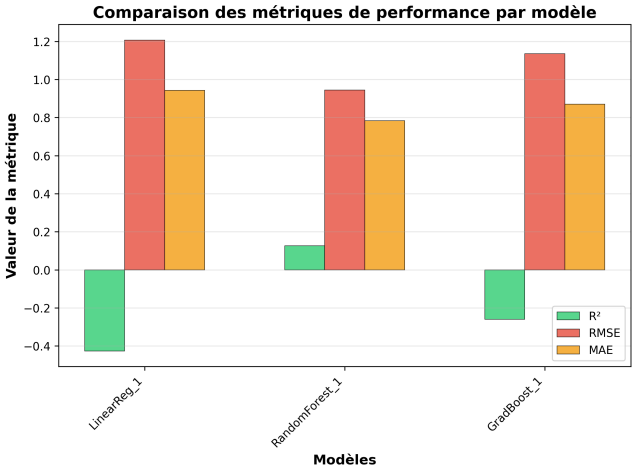

Figure 6.3 : Comparaison des métriques par modèle

6.4 Analyse des résultats

Résultats très limités

Les résultats sont clairement insuffisants pour constituer un modèle prédictif utilisable.

- **Régression linéaire** ($R^2 = -0{,}43$) **:** le modèle prédit moins bien que la simple moyenne des notes, signe d'un problème de linéarité ou de sur-apprentissage.
- **Gradient Boosting** ($R^2 = -0{,}26$) **:** même constat.
- **Random Forest** ($R^2 = +0{,}13$) **:** seul modèle légèrement positif, mais avec un RMSE de 0,95 point, la prédiction reste très imprécise.

La meilleure performance du Random Forest s'explique par sa capacité à modéliser des relations non linéaires et des interactions entre variables, sans nécessiter les hypothèses strictes de la régression linéaire. Cependant, avec seulement 95 observations et une vingtaine de variables, le risque de sur-apprentissage est élevé.

Les graphiques de prédictions vs valeurs réelles confirment ce diagnostic : les points sont largement dispersés autour de la diagonale, sans tendance claire.

Chapitre 7

Discussion, limites et perspectives

7.1 Pourquoi ces résultats sont-ils décevants ?

7.1.1 Taille insuffisante du jeu de données

Avec **95 étudiants** pour une vingtaine de variables, le jeu de données est très petit au regard des standards du Machine Learning. La règle empirique recommande au minimum 10 à 20 observations par variable explicative. Ici, le ratio est à peine de 4 à 5, ce qui favorise le sur-apprentissage (diminuer le nombre de variables sélectionnées n'améliore toutefois pas les scores).

7.1.2 Questionnement sur la pertinence des variables

Les features extraites des logs capturent une présence *quantitative* sur la plateforme, mais pas la qualité du travail fourni, ni les facteurs extérieurs à la plateforme.

Or, la réussite scolaire est un phénomène multifactoriel :

— qualité du travail personnel hors ligne ;

— capacités cognitives individuelles ;

— contexte socio-économique ;

— présence en cours, participation aux TD/TP ;

— motivation, état émotionnel, santé.

Un étudiant peu actif sur ARCHE peut très bien réussir en travaillant intensivement sur ses supports papier. Les traces numériques ne capturent qu'une infime partie des facteurs explicatifs.

7.1.3 Absence d'optimisation des hyperparamètres

Par manque de temps, la recherche d'hyperparamètres optimaux (via `GridSearchCV` ou `RandomizedSearchCV`) n'a pas été réalisée. La validation croisée (*cross-validation*), qui permettrait une estimation plus robuste des performances, n'a pas non plus été mise en oeuvre.

7.1.4 Contraintes techniques

L'interdiction des notebooks Jupyter a complexifié la phase d'exploration et d'itération rapide. Les notebooks sont l'outil standard pour le travail exploratoire en data science (CRISP-DM), permettant de visualiser rapidement les données et d'affiner le feature engineering de façon interactive.

7.2 Une classification aurait peut-être été plus adaptée

Compte tenu des données disponibles et de l'objectif pratique du projet (identifier les étudiants en difficulté), une approche de **classification** aurait été plus pertinente qu'une régression.

L'idée serait de transformer la variable cible :

$$\text{classe} = \begin{cases} \text{À risque} & \text{si note} < 10 \\ \text{En réussite} & \text{si note} \geq 10 \end{cases} \tag{7.1}$$

Cette approche présente plusieurs avantages :

— une cible binaire est généralement plus facile à prédire qu'une valeur continue sur 20 ;

— les métriques de classification (précision, rappel, F1-score, AUC-ROC) sont plus interprétables dans un contexte pédagogique ;

— l'asymétrie des erreurs peut être modélisée explicitement : une fausse alarme est moins grave qu'un étudiant à risque non détecté.

Cette évolution est envisagée pour la soutenance orale.

7.3 Pistes d'amélioration

1. **Passer à la classification** avec `SVC` (petit dataset), `RandomForestClassifier` ou `GradientBoostingClassifier`.
2. **Enrichir les données** en croisant avec d'autres sources (assiduité en présentiel, résultats intermédiaires, profil étudiant) pour obtenir des variables plus prédictives.
3. **Optimiser les hyperparamètres** via une recherche randomisée avec validation croisée stratifiée.
4. **Approfondir le feature engineering :**
   — tendance temporelle (l'activité augmente-t-elle avant les examens ?) ;
   — ratio soumissions / consultations (engagement actif vs passif).
5. **Augmenter le jeu de données** en agrégeant plusieurs promotions ou plusieurs cours pour atteindre une taille d'échantillon suffisante.

Conclusion

Ce projet a permis de mettre en oeuvre un pipeline complet de data science sur un jeu de données réel, du chargement jusqu'à la comparaison de modèles, dans le respect des contraintes imposées (interface Tkinter, sans notebook Jupyter).

Les résultats obtenus sont frustrants et décevants d'un point de vue prédictif : aucun des trois modèles testés ne parvient à prédire la note d'un étudiant de façon fiable. Cela tient à la faible taille du jeu de données, à la nature limitée des informations disponibles dans les logs, et à l'absence d'optimisation systématique des hyperparamètres.

Ces constats illustrent une réalité fréquente en data science : disposer d'un pipeline technique robuste ne suffit pas. La **qualité et la pertinence des données** sont le facteur déterminant de la performance d'un modèle prédictif.

La piste la plus prometteuse pour la suite serait de reformuler le problème comme une tâche de **classification** (étudiant à risque vs en réussite), plus robuste statistiquement et plus actionnable pédagogiquement.